```C++
srand(time(0));
```

Also make sure you include:

```C++
#include <ctime>
```

💡 Why this works:

- `time(0)` gives current time (changes every second)
- So every run → different seed → different random numbers → different roast

---

📌 In short:

> Without `srand()` → same output every time
> With `srand(time(0))` → different output every run

If you're testing and still seeing same output, check:

- You didn't accidentally remove `srand(time(0))`
- You're not running the program multiple times **within the same second** (rare, but possible)

What do you want to tweak next? 😏

and inside `main()` (before using `rand()`):

```cpp
srand(time(0));
```

🔧 Final relevant part:

```cpp
srand(time(0));   // seed

int index = rand() % roasts.size();
```

💡 Why this works:

- `time(0)` changes every second
- So every run → new seed → different random numbers → different roast

---

⚠️ Small tip:

If you run the program multiple times **within the same second**, you *might* still see the same output. Totally normal.

That's it. Fixed 👍

3. Why `srand(time(0))` is important

```
C++
srand(time(0));
```

- This sets the **starting point (seed)** for random numbers
- `time(0)` changes every second → so every run gets a new seed

👉 Without this:

- `rand()` will generate the **same sequence every time**
- So same roast again and again 🤯

👉 With this:

- Different sequence → different roast each run

🔄 Flow in your program:

1. User enters name
2. `srand(time(0))` sets randomness
3. `rand()` generates number
4. `% size` converts it to valid index
5. That roast is picked
6. `{name}` gets replaced
7. Final roast is printed

If you want to sound smart in viva, just say:

> "rand() generates pseudo-random numbers, and modulo is used to map them within the vector range. srand(time(0)) ensures different outputs across runs."

Bas prof impressed 😎

ChatGPT - Movie Ticket Booking

Can you explain how the random selection works in this program and how rand() is used here?

Yeah, good question — this is the part most people just copy without understanding.

🧠 **How random selection works here:**

1. `rand()`
   - `rand()` generates a **pseudo-random number** (not truly random, but looks random)
   - It gives a big integer (like 1804289383 etc.)

---

2. Limiting it to your vector size

```cpp
int index = rand() % roasts.size();
```

- `roasts.size()` = total number of roast templates (say 10)
- `%` (modulo) makes sure the result is between:

```
0 to 9
```

- So now `index` becomes a **valid position in the vector**

👉 Example:

- `rand()` → 57
- `57 % 10 = 7`
- So roast at index 7 is selected

---

3. Why `srand(time(0))` is important

ChatGPT - Movie Ticket Booking

How can I fix this issue so that the program generates different roast outputs every time?

Easy fix.

You just need to **seed the random number generator** so it doesn't start from the same point every time.

✅ Do this:

Add these two things:

```cpp
#include <ctime>
```

and inside `main()` (before using `rand()`):

```cpp
srand(time(0));
```

🔧 Final relevant part:

```cpp
srand(time(0));   // seed

int index = rand() % roasts.size();
```

💡 Why this works:

- `time(0)` changes every second

ChatGPT - Movie Ticket Booking

My program is generating the same roast every time I run it. I am using rand(). Why is this happening?

That's actually a very common issue.

The reason is: `rand()` **is deterministic** unless you change its starting point (seed).

By default, if you don't set a seed, C++ uses the **same seed every time the program runs**, so `rand()` generates the **same sequence of numbers** → same index → same roast.

So even though it *looks* random, it's repeating the same pattern every run.

Example of what's happening:

- Run 1 → `rand()` gives 57 → roast 7
- Run 2 → again 57 → same roast

✅ How to fix it

You need to add this line **once in your program**:

```cpp
srand(time(0));
```

Also make sure you include:

```cpp
#include <ctime>
```

💡 Why this works:

ChatGPT - Movie Ticket Booking

Write a simple and beginner-friendly C++ console program that generates funny roast messages.

The program should:
- Ask the user to enter their name
- Store at least 10 different roast templates in a vector
- Each template should contain a placeholder {name}
- Randomly select one roast each time the program runs
- Replace {name} with the actual user input
- Display the final roast message

Make sure the code is easy to understand and include basic comments.

Alright, here's a clean, beginner-friendly C++ program for your roast generator:

C++

```cpp
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
using namespace std;

int main() {
    string name;

    // Take user input
    cout << "Enter your name: ";
    cin >> name;

    // Store roast templates
    vector<string> roasts = {
        "{name} writes code so slow that even a turtle switched to Python.",
        "If procrastination were an Olympic sport, { } would win gold.",
        "{name}'s debugging style is just staring at the screen.",
        "{name} doesn't debug code — they negotiate with bugs.",
        "{name}'s code has more drama than daily soap operas.",
```

```
    cout << "Enter your name: ";
    cin >> name;

    // Store roast templates
    vector<string> roasts = {
        "{name} writes code so slow that even a turtle switched to Python.",
        "If procrastination were an Olympic sport, {name} would win gold.",
        "{name}'s debugging style is just staring at the screen.",
        "{name} doesn't debug code — they negotiate with bugs.",
        "{name}'s code has more drama than daily soap operas.",
        "{name} runs code once and hopes for the best.",
        "{name}'s logic is still loading... please wait.",
        "Even Google gets confused by {name}'s code.",
        "{name}'s code compiles… emotionally.",
        "{name} and bugs are in a long-term relationship."
    };

    // Seed for randomness
    srand(time(0));

    // Pick random roast
    int index = rand() % roasts.size();
    string roast = roasts[index];

    // Replace {name} with actual name
    size_t pos = roast.find("{name}");
    while (pos != string::npos) {
        roast.replace(pos, 6, name);
        pos = roast.find("{name}");
    }

    // Output
    cout << roast << endl;

    return 0;
}
```

If you run this multiple times, you'll get different roasts because of `srand(time(0))`.

What do you wanna tweak next?

ChatGPT - Movie Ticket Booking

Can you improve the roast templates to make them more creative and funny while keeping them appropriate?

Yeahh this part actually makes your program fun 😄

Here are **better, more creative (but still safe) roast templates** you can directly use:

```cpp
vector<string> roasts = {
    "{name} writes code so slow that even a loading screen gets impatient.",
    "{name}'s code has more twists than a Bollywood plot.",
    "{name} doesn't debug code, they just hope the bug gets bored and leaves.",
    "{name}'s logic is like WiFi in a basement — weak and unreliable.",
    "{name} opens the compiler and immediately forgets what they were doing.",
    "{name}'s code works… but nobody, including {name}, knows how.",
    "{name} treats error messages like suggestions.",
    "{name}'s code is proof that Ctrl+C and Ctrl+V are powerful tools.",
    "{name} writes code like it's a mystery novel — no one knows what's going on.",
    "{name}'s program runs successfully… after emotional support.",
    "{name} spends more time renaming files than writing code.",
    "{name}'s code has commitment issues — it never sticks to one logic.",
    "{name} thinks 'it works on my machine' is a valid excuse.",
    "{name}'s debugging strategy is refreshing the screen and hoping for the best.",
    "{name} writes code so confidently wrong, it almost feels right."
};
```

These are:

- more original
- not offensive
- sound natural (not cringe)
- good for demo / viva

If you want, I can ↳ make them **slightly savage vs s[illegible]r safe version** depending on how strict your teacher is 😏